

WEB FUZZING

CHEAT SHEET

Web Fuzzing Cheatsheet

What is Web Fuzzing?

Web fuzzing is a technique used to discover vulnerabilities, hidden resources, and security issues in web applications by automatically injecting a large set of input data into the application and analyzing its response. The goal is to identify unexpected behaviors or errors that could indicate potential security weaknesses or misconfigurations.

Fuzzing is commonly employed in security testing to find:

- Hidden directories and files
- Insecure APIs and endpoints
- SQL injection points
- Cross-site scripting (XSS) vulnerabilities
- Command injection flaws

Comparison: Brute-Forcing vs. Fuzzing

Criteria	Brute-Forcing	Fuzzing
Definition	Systematically trying all possible combinations of input data to guess a specific value.	Injecting unexpected or random data into an application to find vulnerabilities and hidden resources.
Purpose	Crack passwords, keys, or other access credentials.	Discover application vulnerabilities, hidden files, directories, and input validation issues.
Methodology	Exhaustive search over all possible input combinations.	Dynamic input injection to provoke unexpected application responses.

Criteria	Brute-Forcing	Fuzzing
Focus	Specific input or data, such as passwords or API keys.	General application behavior under various input conditions.
Efficiency	Time-consuming due to exhaustive nature; less efficient for large input spaces.	More efficient in identifying unexpected behaviors and vulnerabilities with varied input.
Tools Used	Password crackers, key recovery tools.	Web fuzzers, vulnerability scanners.
Output	Successful match of the correct input value.	Discovery of vulnerabilities, misconfigurations, and hidden resources.

Miscellaneous Commands

Below are some useful commands that can aid in various tasks related to web fuzzing and testing.

Command	Description
<code>sudo sh -c 'echo "SERVER_IP academy.htb" >> /etc/hosts'</code>	Add a DNS entry for a specific IP address to the <code>/etc/hosts</code> file. This helps resolve domain names locally.
<code>for i in \$(seq 1 1000); do echo \$i >> ids.txt; done</code>	Create a sequence wordlist from 1 to 1000. Useful for brute-forcing numerical IDs or similar patterns.
<code>curl http://admin.academy.htb:PORT/admin/admin.php -X POST -d 'id=key' -H 'Content-Type: application/x-www-form-urlencoded'</code>	Use <code>curl</code> to send a POST request with specific data and headers, simulating form submissions or API calls.

Commonly Used SecLists Wordlists

SecLists is a collection of multiple types of wordlists used by security researchers and penetration testers. Below is a table of some commonly used wordlists from SecLists, which can be incredibly valuable during web fuzzing.

Wordlist	Description
<code>/usr/share/seclists/Discovery/Web-Content/common.txt</code>	General-Purpose Wordlist: Contains a broad range of common directory and file names on web servers. It's an excellent starting point for fuzzing and often yields valuable results.
<code>/usr/share/seclists/Discovery/Web-Content/directory-list-2.3-medium.txt</code>	Directory-Focused Wordlist: A more extensive wordlist specifically focused on directory names. It's a good choice when you need a deeper dive into potential directories.
<code>/usr/share/seclists/Discovery/Web-Content/raft-large-directories.txt</code>	Large Directory Wordlist: Boasts a massive collection of directory names compiled from various sources. It's a valuable resource for thorough fuzzing campaigns.
<code>/usr/share/seclists/Discovery/Web-Content/big.txt</code>	Comprehensive Wordlist: A massive wordlist containing both directory and file names. Useful when you want to cast a wide net and explore all possibilities.

Tips for Using Wordlists Effectively

Tip	Explanation
Choose the Right Wordlist	Select wordlists relevant to the target environment and technology stack for better results.
Combine Wordlists	Use multiple wordlists together to increase the breadth of your fuzzing efforts.
Customize Wordlists	Modify existing wordlists or create your own based on specific knowledge about the target.

Tip	Explanation
Monitor Performance	Large wordlists can be resource-intensive; monitor performance and adjust as needed.
Leverage Community Resources	Utilize community-maintained wordlists for the latest and most effective fuzzing strategies.

Tools for Web Fuzzing

ffuf (Fuzz Faster U Fool)

ffuf is a fast web fuzzer written in Go that allows you to discover directories and files on web servers.

Command	Description
<code>ffuf -u http://example.com/FUZZ</code>	Basic fuzzing of a URL path.
<code>ffuf -u http://example.com/FUZZ -w wordlist.txt</code>	Fuzz with a specific wordlist.
<code>ffuf -u http://example.com/FUZZ -w wordlist.txt -ic</code>	Fuzz with a specific wordlist, automatically ignoring any comments in the wordlist.
<code>ffuf -u http://example.com/FUZZ -w wordlist.txt -c</code>	Colorize the output for better readability.
<code>ffuf -u http://example.com/FUZZ -w wordlist.txt -mc 200</code>	Filter results by status code (e.g., 200).
<code>ffuf -u http://example.com/FUZZ -w wordlist.txt -mr "Welcome"</code>	Filter results by matching a regex pattern.
<code>ffuf -u http://example.com/FUZZ -w wordlist.txt -e .php,.html</code>	Add extensions to each wordlist entry.
<code>ffuf -u http://example.com/FUZZ -w wordlist.txt -t 50</code>	Set the number of threads (e.g., 50) for faster fuzzing.
<code>ffuf -u http://example.com/FUZZ -w wordlist.txt -x http://127.0.0.1:8080</code>	Use a proxy for requests.

gobuster

gobuster is a tool used to brute-force URIs (directories and files) in web sites and DNS subdomains.

Command	Description
<code>gobuster dir -u http://example.com -w wordlist.txt</code>	Directory fuzzing using a wordlist.
<code>gobuster dir -u http://example.com -w wordlist.txt -x .php,.html</code>	Fuzz with specific extensions.
<code>gobuster dir -u http://example.com -w wordlist.txt -s 200</code>	Filter results by status code (e.g., 200).
<code>gobuster dir -u http://example.com -w wordlist.txt -t 50</code>	Set the number of concurrent threads (e.g., 50).
<code>gobuster dir -u http://example.com -w wordlist.txt -o results.txt</code>	Output results to a file.
<code>gobuster dns -d example.com -w subdomains.txt</code>	Fuzz DNS subdomains using a wordlist.
<code>gobuster dns -d example.com -w subdomains.txt -i</code>	Show IP addresses of discovered subdomains.
<code>gobuster dns -d example.com -w subdomains.txt -z</code>	Silent mode; suppress output except for results.

wenum (Wfuzz Fork)

wenum is a fork of **wfuzz**, a versatile web application fuzzer for testing web security.

Command	Description
<code>wenum -c -w wordlist.txt --hc 404 -u http://example.com/FUZZ</code>	Basic fuzzing excluding 404 responses.
<code>wenum -c -w wordlist.txt -d 'username=FUZZ&password=secret' -u http://example.com/login</code>	Fuzz POST data in a form.

Command	Description
<code>wenum -c -w wordlist.txt -b 'session=12345' -u http://example.com/FUZZ</code>	Use a specific cookie for requests.
<code>wenum -c -w wordlist.txt -H 'User-Agent: Wenum' -u http://example.com/FUZZ</code>	Add a custom header to requests.
<code>wenum -c -w wordlist.txt -t 50 -u http://example.com/FUZZ</code>	Set the number of threads (e.g., 50) for faster fuzzing.
<code>wenum -c -w wordlist.txt -X PUT -u http://example.com/FUZZ</code>	Fuzz using a specific HTTP method (e.g., PUT).
<code>wenum -c -w wordlist.txt --hs 50 -u http://example.com/FUZZ</code>	Filter responses by content length (e.g., 50 bytes).

feroxbuster

feroxbuster is a tool designed for recursive content discovery and web fuzzing.

Command	Description
<code>feroxbuster -u http://example.com -w wordlist.txt</code>	Basic URL fuzzing with a wordlist.
<code>feroxbuster -u http://example.com -w wordlist.txt -x</code>	Include specified file extensions in fuzzing.
<code>feroxbuster -u http://example.com -w wordlist.txt -C 404</code>	Exclude responses with status code 404.
<code>feroxbuster -u http://example.com -w wordlist.txt -t 50</code>	Set the number of concurrent threads (e.g., 50).
<code>feroxbuster -u http://example.com -w wordlist.txt --depth 3</code>	Set maximum recursion depth (e.g., 3 levels deep).
<code>feroxbuster -u http://example.com -w wordlist.txt -o results.txt</code>	Save output to a file.
<code>feroxbuster -u http://example.com -w wordlist.txt --no-recursion</code>	Disable recursion into discovered directories.

Command	Description
<code>feroxbuster -u http://example.com -w wordlist.txt --redirect</code>	Follow redirects automatically.

Tips for Effective Web Fuzzing

Tip	Explanation
Use Comprehensive Wordlists	The quality of your wordlist can significantly impact results; choose or create wordlists relevant to the target.
Filter Unwanted Responses	Use status codes or response size filtering to focus on meaningful results and reduce noise.
Adjust Thread Count	Increase thread count for faster fuzzing, but be mindful of server capabilities to avoid overloading.
Monitor Server Responses	Pay attention to anomalies or unexpected behavior in server responses, indicating potential vulnerabilities.
Fuzz with Various HTTP Methods	Test different HTTP methods (GET, POST, PUT, DELETE) to uncover potential vulnerabilities in all endpoints.

Web APIs: REST, SOAP, and GraphQL

What is a Web API?

A **Web API** (Application Programming Interface) is a set of rules and protocols for building and interacting with software applications. APIs allow different applications to communicate with each other over the internet, enabling the integration of various services and data exchange.

Web APIs can be categorized into three main types:

- REST (Representational State Transfer)**
- SOAP (Simple Object Access Protocol)**
- GraphQL**

Each type has its own unique characteristics, advantages, and use cases.

REST (Representational State Transfer)

REST is an architectural style that uses standard HTTP methods to access and manipulate resources on a server. It is known for its simplicity, scalability, and statelessness.

Feature	Description
Protocol	Uses HTTP/HTTPS.
Data Format	Typically JSON, but can also use XML, HTML, or plain text.
Stateless	Each request from a client to a server must contain all the information needed.
CRUD Operations	Uses HTTP methods: GET, POST, PUT, DELETE.
Scalability	Highly scalable due to its stateless nature.
Caching	Supports caching mechanisms to improve performance.
URL Structure	Uses endpoints that represent resources, e.g., <code>/api/users/{id}</code> .
Advantages	Simplicity, flexibility, scalability.
Disadvantages	Can lead to over-fetching or under-fetching data.

REST Fuzzing Tips:

Tip	Explanation
Test All HTTP Methods	Ensure all CRUD operations are tested, as vulnerabilities might exist in any of them.
Validate Input Fields	Fuzz input fields with unexpected data types and formats to uncover validation issues.
Examine Error Messages	Analyze error messages for information disclosure or unintended behavior.
Test Authentication Mechanisms	Check for improper authentication and authorization controls.
Explore API Rate Limits	Test rate limits and throttling controls to ensure the API handles requests properly.
Use Comprehensive Payloads	Leverage a variety of payloads (SQLi, XSS) to test for potential security flaws.

Tip	Explanation
Check Resource Representation	Test different resource representations (JSON, XML) for consistency and security flaws.

SOAP (Simple Object Access Protocol)

SOAP is a protocol for exchanging structured information in web services. It uses XML as its message format and can operate over various protocols like HTTP, SMTP, or TCP.

Feature	Description
Protocol	Protocol-independent but often used with HTTP/HTTPS.
Data Format	Exclusively XML.
Stateful/Stateless	Can be either stateful or stateless.
WS-Security	Built-in security features for message integrity and confidentiality.
Error Handling	Uses specific error codes and messages.
Complexity	More complex due to extensive standards and specifications.
Extensibility	Highly extensible via WS-* standards.
Advantages	Strong security, reliability, and extensibility.
Disadvantages	More complex and less flexible compared to REST.

SOAP Fuzzing Tips:

Tip	Explanation
Analyze WSDL Files	Use WSDL (Web Services Description Language) files to understand the service's operations and inputs.
Validate XML Schema	Test XML inputs against the schema to identify validation flaws.
Check for XML Injection	Fuzz XML data to test for injection vulnerabilities.

Tip	Explanation
Test SOAP Headers	Fuzz SOAP headers to find potential security issues or misconfigurations.
Evaluate WS-Security Implementations	Ensure security implementations are robust and correctly configured.
Test Transport Security	Verify that transport-level security (e.g., HTTPS) is enforced and properly implemented.
Examine SOAP Faults	Analyze SOAP fault messages for potential information leakage.

GraphQL

GraphQL is a query language and runtime for APIs that allows clients to request specific data and define the structure of the response.

Feature	Description
Protocol	Uses HTTP/HTTPS, typically over POST requests.
Data Format	JSON-based queries and responses.
Stateful/Stateless	Stateless architecture.
Query Flexibility	Clients can request exactly what they need, minimizing over-fetching and under-fetching.
Single Endpoint	Typically uses a single endpoint for all operations.
Introspection	Allows clients to query the API schema for available operations and data types.
Advantages	Efficiency, flexibility, and powerful developer tooling.

Disadvantages Potential for complex queries leading to performance issues if not properly managed.

GraphQL Fuzzing Tips:

Tip	Explanation
Test Query Depth and Complexity	Evaluate the server's handling of deeply nested or complex queries to avoid performance bottlenecks.
Validate Input Types and Arguments	Fuzz input arguments with unexpected values and data types to uncover validation flaws.
Examine Query Aliasing and Batching	Test the server's response to aliased queries and batching for potential information leakage.
Check for Introspection Misuse	Ensure introspection is not exposing sensitive information or internal schema details.
Assess Authorization Controls	Verify that access controls are properly enforced for different queries and operations.
Evaluate Rate Limiting	Test rate limits to ensure the API can handle excessive or malicious requests appropriately.
Fuzz Mutations	Mutations can alter data; test for security issues and improper input validation.